

QUESTIONS 10 / 10 completed

Pass Array

PASS ARRAY

Write a program to pass an array to a function using pointers.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 void printArray(int *ptr, int size) {
4     for (int i = 0; i < size; i++) {
5         printf("%d ", *(ptr + i));
6     }
7     printf("\n");
8 }
9
10 int main() {
11     int arr[] = {5, 10, 15, 20, 25};
12     int size = sizeof(arr) / sizeof(arr[0]);
13     printArray(arr, size);
14     return 0;
15 }
```

Structure



STRUCTURE

Write a program to pass a structure to a function using pointers.

```
1 #include <stdio.h>
2
3 struct Student {
4     int rollNo;
5     int marks;
6 };
7
8 void display(struct Student *ptr) {
9     printf("Roll No: %d\n", ptr->rollNo);
10    printf("Marks: %d\n", ptr->marks);
11 }
12
13 int main() {
14     struct Student s = {101, 95};
15     display(&s);
16     return 0;
17 }
```

QUESTIONS 10 / 10 completed

Swap Ptr. ▾

SWAP PTR.

Write a program to swap two variables using a function with pointers.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2
3 void swap(int *a, int *b) {
4     int temp = *a;
5     *a = *b;
6     *b = temp;
7 }
8
9 int main() {
10     int x = 10, y = 20;
11     swap(&x, &y);
12     printf("%d %d\n", x, y);
13     return 0;
14 }
```

QUESTIONS 10 / 10 completed

Max

MAX

Write a program to find the largest element in an array using a function with pointers.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int findMax(int *ptr, int size) {
4     int max = *ptr;
5     for (int i = 1; i < size; i++) {
6         if (*(ptr + i) > max) {
7             max = *(ptr + i);
8         }
9     }
10    return max;
11 }
12
13 int main() {
14     int arr[] = {12, 45, 2, 67, 34};
15     int size = sizeof(arr) / sizeof(arr[0]);
16     printf("%d\n", findMax(arr, size));
17     return 0;
18 }
```

QUESTIONS 10 / 10 completed

Concatenate

CONCATENATE

Write a program to concatenate two strings using a function with pointers.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 void concatenate(char *str1, char *str2) {
4     while (*str1 != '\0') {
5         str1++;
6     }
7     while (*str2 != '\0') {
8         *str1 = *str2;
9         str1++;
10        str2++;
11    }
12    *str1 = '\0';
13 }
14
15 int main() {
16     char str1[50] = "Hello ";
17     char str2[] = "World";
18     concatenate(str1, str2);
19     printf("%s\n", str1);
20     return 0;
21 }
```

QUESTIONS 10 / 10 completed

Count O/E

COUNT O/E

Write a program to count the number of even and odd elements in an array using a function with pointers.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 void countEvenOdd(int *arr, int size, int *even, int *odd) {
4     *even = 0;
5     *odd = 0;
6     for (int i = 0; i < size; i++) {
7         if (*(arr + i) % 2 == 0) {
8             (*even)++;
9         } else {
10             (*odd)++;
11         }
12     }
13 }
14
15 int main() {
16     int arr[] = {1, 2, 3, 4, 5, 6};
17     int size = sizeof(arr) / sizeof(arr[0]);
18     int evenCount, oddCount;
19
20     countEvenOdd(arr, size, &evenCount, &oddCount);
21 }
```


QUESTIONS 10 / 10 completed

Sort

SORT

Write a program to check if a given array is sorted using a function with pointers.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int isSorted(int *ptr, int size) {
4     for (int i = 0; i < size - 1; i++) {
5         if (*(ptr + i) > *(ptr + i + 1)) {
6             return 0;
7         }
8     }
9     return 1;
10 }
11
12 int main() {
13     int arr[] = {10, 20, 30, 40, 50};
14     int size = sizeof(arr) / sizeof(arr[0]);
15
16     if (isSorted(arr, size)) {
17         printf("Sorted\n");
18     } else {
19         printf("Not Sorted\n");
20     }
21 }
```

QUESTIONS 10 / 10 completed

Reverse

REVERSE

Write a program to reverse an array using a function with pointers.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 void reverseArray(int *ptr, int size) {
4     int *start = ptr;
5     int *end = ptr + size - 1;
6     while (start < end) {
7         int temp = *start;
8         *start = *end;
9         *end = temp;
10        start++;
11        end--;
12    }
13 }
14
15 int main() {
16     int arr[] = {1, 2, 3, 4, 5};
17     int size = sizeof(arr) / sizeof(arr[0]);
18
19     reverseArray(arr, size);
20
21     for (int i = 0; i < size; i++) {
```


QUESTIONS 10 / 10 completed

Copy String

COPY STRING

Write a program to copy one string to another using a function with pointers.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 void copyString(char *dest, char *src) {
4     while (*src != '\0') {
5         *dest = *src;
6         src++;
7         dest++;
8     }
9     *dest = '\0';
10 }
11
12 int main() {
13     char source[] = "Hello";
14     char destination[20];
15
16     copyString(destination, source);
17     printf("%s\n", destination);
18
19     return 0;
20 }
```

QUESTIONS 10 / 10 completed

Sort Array



SORT ARRAY

Write a program to sort an array in ascending order using a function with pointers.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 void sortArray(int *ptr, int size) {
4     for (int i = 0; i < size - 1; i++) {
5         for (int j = i + 1; j < size; j++) {
6             if (*(ptr + i) > *(ptr + j)) {
7                 int temp = *(ptr + i);
8                 *(ptr + i) = *(ptr + j);
9                 *(ptr + j) = temp;
10            }
11        }
12    }
13 }
14
15 int main() {
16     int arr[] = {40, 10, 50, 20, 30};
17     int size = sizeof(arr) / sizeof(arr);
18
19     sortArray(arr, size);
20
21     for (int i = 0; i < size; i++) {
```